

EXPERIMENT 3

Title

Create an Employee-Department-Project schema, insert at least 30 employees across 5 departments and 8 projects, and demonstrate SQL selection, projection, aggregate functions, GROUP BY, HAVING, CASE expressions, and ORDER BY.

Aim

To design and implement a relational database for employees, departments and projects, populate it with sample data, and execute SQL queries using commonly used relational and aggregate operations.

Theory

A relational database stores data in tables that are connected through keys. In this experiment, the Department table stores department information, the Employee table stores employee information, and the Project table stores project information. An Employee-Project relationship is represented using the Employee_Project table because one employee can work on multiple projects and one project can have multiple employees.

- Selection (WHERE): filters rows that satisfy a condition.
- Projection (SELECT): retrieves only the required columns from a table.
- Aggregate functions: perform calculations such as COUNT(), SUM(), AVG(), MIN() and MAX().
- GROUP BY: groups rows having the same value so aggregate calculations can be performed for each group.
- HAVING: filters groups after GROUP BY and is commonly used with aggregate functions.
- CASE expression: applies conditional logic and returns different values based on conditions.
- ORDER BY: sorts the query result in ascending (ASC) or descending (DESC) order.
- Primary Key uniquely identifies a row, while Foreign Key maintains relationships between tables.

Database Schema

```
DEPARTMENT(  
    department_id INT PRIMARY KEY,  
    department_name VARCHAR(50) UNIQUE,  
    location VARCHAR(50)  
)
```

```
EMPLOYEE(  
    employee_id INT PRIMARY KEY,  
    employee_name VARCHAR(60),  
    job_title VARCHAR(50),  
    salary DECIMAL(10,2),  
    hire_date DATE,  
    department_id INT,  
    FOREIGN KEY (department_id) REFERENCES DEPARTMENT(department_id)  
)
```

```
PROJECT(
    project_id INT PRIMARY KEY,
    project_name VARCHAR(80),
    budget DECIMAL(12,2),
    start_date DATE,
    department_id INT,
    FOREIGN KEY (department_id) REFERENCES DEPARTMENT(department_id)
)
```

```
EMPLOYEE_PROJECT(
    employee_id INT,
    project_id INT,
    hours_worked INT,
    role VARCHAR(50),
    PRIMARY KEY (employee_id, project_id),
    FOREIGN KEY (employee_id) REFERENCES EMPLOYEE(employee_id),
    FOREIGN KEY (project_id) REFERENCES PROJECT(project_id)
)
```

SQL Implementation

1. Create Database

```
CREATE DATABASE company_db;
USE company_db;
```

2. Create Tables

```
CREATE TABLE Department (
    department_id INT PRIMARY KEY,
    department_name VARCHAR(50) UNIQUE NOT NULL,
    location VARCHAR(50)
);
```

```
CREATE TABLE Employee (
    employee_id INT PRIMARY KEY,
    employee_name VARCHAR(60) NOT NULL,
    job_title VARCHAR(50),
    salary DECIMAL(10,2),
    hire_date DATE,
    department_id INT,
    FOREIGN KEY (department_id) REFERENCES Department(department_id)
);
```

```
CREATE TABLE Project (
    project_id INT PRIMARY KEY,
    project_name VARCHAR(80) NOT NULL,
    budget DECIMAL(12,2),
    start_date DATE,
    department_id INT,
    FOREIGN KEY (department_id) REFERENCES Department(department_id)
);
```

```
CREATE TABLE Employee_Project (
    employee_id INT,
    project_id INT,
    hours_worked INT,
    role VARCHAR(50),
    PRIMARY KEY (employee_id, project_id),
    FOREIGN KEY (employee_id) REFERENCES Employee(employee_id),
```

```
FOREIGN KEY (project_id) REFERENCES Project(project_id)
);
```

3. Insert 5 Departments

```
INSERT INTO Department VALUES
(1, 'Engineering', 'Delhi'),
(2, 'Human Resources', 'Mumbai'),
(3, 'Finance', 'Bengaluru'),
(4, 'Marketing', 'Pune'),
(5, 'Operations', 'Hyderabad');
```

4. Insert 8 Projects

```
INSERT INTO Project VALUES
(101, 'AI Analytics Platform', 1500000, '2026-01-15', 1),
(102, 'Cloud Migration', 1200000, '2026-02-01', 1),
(103, 'Employee Wellness', 450000, '2026-01-20', 2),
(104, 'Budget Automation', 800000, '2026-03-01', 3),
(105, 'Digital Campaign', 650000, '2026-02-15', 4),
(106, 'Customer Insights', 900000, '2026-03-10', 4),
(107, 'Supply Optimization', 1100000, '2026-01-25', 5),
(108, 'Process Automation', 1000000, '2026-03-20', 5);
```

5. Insert 30 Employees

```
INSERT INTO Employee VALUES
(1001, 'Aarav Sharma', 'Software Engineer', 85000, '2023-06-12', 1),
(1002, 'Priya Verma', 'Data Analyst', 78000, '2024-01-15', 1),
(1003, 'Rahul Mehta', 'Cloud Engineer', 92000, '2022-08-20', 1),
(1004, 'Sneha Kapoor', 'AI Engineer', 98000, '2023-11-05', 1),
(1005, 'Vikram Singh', 'DevOps Engineer', 88000, '2024-04-18', 1),
(1006, 'Neha Gupta', 'Software Engineer', 82000, '2025-02-10', 1),

(1007, 'Riya Nair', 'HR Executive', 60000, '2023-03-14', 2),
(1008, 'Karan Joshi', 'HR Manager', 90000, '2021-07-22', 2),
(1009, 'Pooja Iyer', 'Recruiter', 62000, '2024-05-09', 2),
(1010, 'Amit Rao', 'Training Specialist', 68000, '2022-10-11', 2),
(1011, 'Simran Kaur', 'HR Analyst', 65000, '2025-01-17', 2),
(1012, 'Nitin Das', 'HR Executive', 59000, '2024-08-26', 2),

(1013, 'Aditya Shah', 'Financial Analyst', 76000, '2022-02-18', 3),
(1014, 'Meera Jain', 'Accountant', 68000, '2023-09-07', 3),
(1015, 'Rohan Malhotra', 'Finance Manager', 95000, '2021-05-30', 3),
(1016, 'Kavya Rao', 'Risk Analyst', 80000, '2024-06-21', 3),
(1017, 'Sahil Bansal', 'Accountant', 67000, '2025-03-13', 3),
(1018, 'Ishita Roy', 'Budget Analyst', 73000, '2023-12-01', 3),

(1019, 'Arjun Sethi', 'Marketing Executive', 62000, '2023-04-25', 4),
(1020, 'Ananya Bose', 'Marketing Manager', 93000, '2021-11-19', 4),
(1021, 'Dev Patel', 'SEO Specialist', 65000, '2024-02-08', 4),
(1022, 'Tanya Arora', 'Content Strategist', 70000, '2023-07-16', 4),
(1023, 'Manish Kumar', 'Brand Analyst', 68000, '2025-01-29', 4),
(1024, 'Nisha Yadav', 'Marketing Executive', 61000, '2024-09-12', 4),

(1025, 'Mohit Sharma', 'Operations Manager', 94000, '2021-06-15', 5),
(1026, 'Ayesha Khan', 'Operations Analyst', 72000, '2023-10-03', 5),
(1027, 'Harsh Vardhan', 'Process Engineer', 81000, '2022-12-14', 5),
(1028, 'Divya Menon', 'Supply Analyst', 75000, '2024-03-27', 5),
(1029, 'Yash Thakur', 'Operations Executive', 64000, '2025-04-05', 5),
(1030, 'Komal Singh', 'Process Analyst', 70000, '2023-08-31', 5);
```

6. Assign Employees to Projects

```
INSERT INTO Employee_Project VALUES
(1001,101,180,'Developer'), (1002,101,160,'Data Analyst'),
(1003,102,175,'Cloud Engineer'), (1004,101,190,'AI Engineer'),
(1005,102,165,'DevOps Engineer'), (1006,101,150,'Developer'),
(1007,103,120,'HR Coordinator'), (1008,103,140,'Project Lead'),
(1009,103,110,'Recruiter'), (1010,103,125,'Trainer'),
(1011,103,100,'HR Analyst'), (1012,103,95,'HR Executive'),
(1013,104,155,'Financial Analyst'), (1014,104,145,'Accountant'),
(1015,104,170,'Project Lead'), (1016,104,135,'Risk Analyst'),
(1017,104,120,'Accountant'), (1018,104,130,'Budget Analyst'),
(1019,105,115,'Marketing Executive'), (1020,105,160,'Project Lead'),
(1021,105,125,'SEO Specialist'), (1022,106,145,'Content Strategist'),
(1023,106,120,'Brand Analyst'), (1024,106,110,'Marketing Executive'),
(1025,107,175,'Project Lead'), (1026,107,150,'Operations Analyst'),
(1027,108,160,'Process Engineer'), (1028,107,140,'Supply Analyst'),
(1029,108,115,'Operations Executive'), (1030,108,130,'Process Analyst');
```

SQL Queries Demonstrating Required Concepts

A. Selection (WHERE)

```
SELECT *
FROM Employee
WHERE salary > 80000;
```

B. Projection (SELECT)

```
SELECT employee_id, employee_name, job_title
FROM Employee;
```

C. Selection + Projection

```
SELECT employee_name, salary
FROM Employee
WHERE department_id = 1;
```

D. Aggregate Functions

```
SELECT
    COUNT(*) AS total_employees,
    AVG(salary) AS average_salary,
    MIN(salary) AS minimum_salary,
    MAX(salary) AS maximum_salary,
    SUM(salary) AS total_salary
FROM Employee;
```

E. GROUP BY

```
SELECT department_id, COUNT(*) AS employee_count,
    AVG(salary) AS average_salary
FROM Employee
GROUP BY department_id;
```

F. GROUP BY with JOIN

```
SELECT d.department_name,
    COUNT(e.employee_id) AS employee_count,
    AVG(e.salary) AS average_salary
FROM Department d
LEFT JOIN Employee e
    ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name;
```

G. HAVING

```
SELECT department_id, AVG(salary) AS average_salary
FROM Employee
GROUP BY department_id
HAVING AVG(salary) > 75000;
```

H. CASE Expression

```
SELECT employee_name, salary,
       CASE
           WHEN salary >= 90000 THEN 'High Salary'
           WHEN salary >= 70000 THEN 'Medium Salary'
           ELSE 'Low Salary'
       END AS salary_category
FROM Employee;
```

I. CASE with Department

```
SELECT employee_name, department_id,
       CASE department_id
           WHEN 1 THEN 'Engineering'
           WHEN 2 THEN 'Human Resources'
           WHEN 3 THEN 'Finance'
           WHEN 4 THEN 'Marketing'
           WHEN 5 THEN 'Operations'
           ELSE 'Unknown'
       END AS department_name
FROM Employee;
```

J. ORDER BY Ascending

```
SELECT employee_name, salary
FROM Employee
ORDER BY salary ASC;
```

K. ORDER BY Descending

```
SELECT employee_name, salary
FROM Employee
ORDER BY salary DESC;
```

L. GROUP BY + HAVING + ORDER BY

```
SELECT d.department_name,
       COUNT(e.employee_id) AS employee_count,
       AVG(e.salary) AS average_salary
FROM Department d
JOIN Employee e
    ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name
HAVING COUNT(e.employee_id) >= 5
ORDER BY average_salary DESC;
```

Sample Verification Queries

-- Check departments

```
SELECT * FROM Department;
```

-- Check all employees

```
SELECT * FROM Employee;
```

-- Check projects

```
SELECT * FROM Project;
```

```
-- Check employee-project assignments
SELECT * FROM Employee_Project;
```

Expected Outcome

The database is successfully created with 5 departments, 8 projects and 30 employees. The SQL queries demonstrate selection, projection, aggregate functions, GROUP BY, HAVING, CASE expressions and ORDER BY. The Employee_Project table also demonstrates the many-to-many relationship between employees and projects.

Result

Thus, the Employee-Department-Project relational schema was created and populated successfully. The required SQL operations were executed to retrieve, summarize, classify and sort the stored data.